

LAPORAN PRAKTIKUM
STRUKTUR DATA
“ DLL – DOUBLE LINKED LIST”



disusun Oleh:
FILZI JELILA INDA ROBBANI
(2511533019)

Dosen Pengampu:
Dr. WAHYUDI, S.T, M.T
Asisten Praktikum:
ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadiran Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Struktur Data kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini saya susun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Struktur Data di Universitas Andalas. Laporan ini diharapkan dapat menambah wawasan mengenai penggunaan struktur data Double Linked List pada Java .

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan.....	5
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Langkah Kerja Praktikum	6
2.1.1 Membuat kelas pertama (NodeDLL).....	6
2.1.2 Membuat kelas kedua (InsertDLL)	7
2.1.3 Membuat program ketiga (PenelusuranDLL).....	9
2.1.4 Membuat program keempat (HapusDLL).....	11
2.2 Tugas Praktikum	15
2.2.1 Deskripsi Tugas.....	15
2.2.2 Kelas Lagu	15
2.2.3 Kelas Musik	16
BAB III PENUTUP	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya, telah dipelajari mengenai struktur data Queue yang digunakan untuk mengelola data secara terurut dengan prinsip FIFO (First In First Out). Queue membantu mahasiswa memahami bagaimana data diproses sesuai urutan masuk dan keluar, serta bagaimana pengelolaan data dilakukan secara terstruktur dalam pemrograman Java. Selain itu, praktikum tersebut juga memberikan pemahaman mengenai penggunaan array dalam implementasi struktur data sederhana.

Selanjutnya, pada praktikum kali ini dilakukan pengembangan materi dengan mempelajari Single Linked List (SLL) menggunakan bahasa pemrograman Java. Single Linked List merupakan struktur data linear yang terdiri dari kumpulan node yang saling terhubung melalui pointer. Setiap node memiliki dua bagian utama, yaitu data dan alamat menuju node berikutnya. Berbeda dengan array yang memiliki ukuran tetap, Single Linked List bersifat dinamis sehingga penambahan maupun penghapusan data dapat dilakukan dengan lebih fleksibel.

Dalam praktikum ini dilakukan beberapa operasi dasar pada Single Linked List, seperti traversal atau penelusuran data, pencarian data, penambahan node di awal, tengah, dan akhir list, serta penghapusan node pada posisi tertentu. Melalui implementasi tersebut, mahasiswa dapat memahami cara kerja pointer dan hubungan antar node dalam struktur data dinamis.

Dengan demikian, praktikum ini menjadi lanjutan yang penting untuk memperkuat pemahaman mahasiswa mengenai struktur data linear, khususnya dalam pengelolaan data yang lebih fleksibel, dinamis, dan efisien menggunakan konsep linked list berbasis objek.

1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

- a. Memahami konsep dasar Single Linked List dalam Java.
- b. Memahami penggunaan node dan pointer pada linked list.
- c. Membuat program penelusuran data pada linked list.
- d. Membuat program penambahan node pada linked list.
- e. Membuat program penghapusan node pada linked list.
- f. Mengimplementasikan Single Linked List menggunakan Java..
- g. Melatih kemampuan dalam mengimplementasikan konsep teori struktur data ke dalam program Java.

1.3 Manfaat

- a. Mahasiswa dapat memahami penggunaan struktur data dinamis dalam Java.
- b. Mahasiswa dapat memahami bagaimana hubungan antar node pada linked list.
- c. Meningkatkan kemampuan dalam mengelola data secara efisien dan fleksibel.
- d. Menambah pemahaman mengenai operasi dasar linked list seperti traversal, insertion, searching, dan deletion.
- e. Meningkatkan kemampuan problem solving dalam pemrograman.

BAB II

PEMBAHASAN

2.1 Langkah Kerja Praktikum

2.1.1 Membuat kelas pertama (NodeDLL)

- 1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah package baru dengan nama pekan6_nim terus pilih “New”, pilih class. Buat nama class “NodeDLL_2511533019”.
- 2) Lalu, masukkan syntax seperti berikut.

```
1) package Pekan6_2511533019;  
2)  
3) public class NodeDLL_2511533019 {  
4)     // TODO Auto-generated method stub  
5)     //mendefinisikan kelas node  
6)     int data_3019;//data  
7)     NodeDLL_2511533019 next_3019;//pointer ke  
    next node  
8)     NodeDLL_2511533019 prev_3019;// pointer ke  
    previous node  
9)  
10)    //konstruktor  
11)    public NodeDLL_2511533019(int data_3019) {  
12)        this.data_3019=data_3019;  
13)        this.next_3019=null;  
14)        this.prev_3019=null;  
15)    }  
16) }
```

Program pertama yaitu NodeDLL_2511533019 digunakan untuk mendefinisikan node pada struktur data Double Linked List (DLL). Di dalam kelas ini terdapat tiga atribut, yaitu data_3019 untuk menyimpan nilai data, next_3019 sebagai pointer yang menunjuk ke node berikutnya, dan prev_3019 sebagai pointer yang menunjuk ke node sebelumnya.

Kelas ini juga memiliki konstruktor NodeDLL_2511533019(int data_3019) yang berfungsi menginisialisasi data node saat objek dibuat. Ketika node baru dibuat, pointer next_3019 dan prev_3019 otomatis bernilai null karena node belum terhubung dengan node lain..

2.1.2 Membuat kelas kedua (InsertDLL)

- 1) Klik kanan package pekan6 dan pilih “New”, pilih class. Lalu buat class baru dengan nama “InsertDLL_2511533019.”
- 2) Masukkan syntax seperti berikut.

```
1) package Pekan6_2511533019;
2)
3) public class InsertDLL_2511533019 {
4)     //menambahkan node diawal DLL
5)     static NodeDLL_2511533019
6)     insertBegin_3019(NodeDLL_2511533019 head_3019, int
7)     data_3019) {
8)         //buat node baru
9)         NodeDLL_2511533019 new_node = new
10)         NodeDLL_2511533019(data_3019);
11)         //jadikan pointer nextnya head
12)         new_node.next_3019 = head_3019;
13)         //jadikan pointer prehead ke new_node
14)         if (head_3019!= null) {
15)             head_3019.prev_3019 = new_node;
16)         }
17)         return new_node;
18)     }
19)     //fungsi menambahkan node di akhir
20)     public static NodeDLL_2511533019 insertEnd_3019
21)     (NodeDLL_2511533019 head_3019, int newData_3019) {
22)         //buat node baru
23)         NodeDLL_2511533019 newNode_3019 = new
24)         NodeDLL_2511533019 (newData_3019);
25)         //jika DLL null jadikan head
26)         if(head_3019 == null) {
27)             head_3019 = newNode_3019;
28)         }else{
29)             NodeDLL_2511533019 curr_3019 = head_3019;
30)             while (curr_3019.next_3019!= null) {
31)                 curr_3019 = curr_3019.next_3019;
32)             }
33)             curr_3019.next_3019 = newNode_3019;
34)             newNode_3019.prev_3019 = curr_3019;
35)         }
36)         return head_3019;
37)     }
38) }
39) //fungsi menambahkan node di posisi tertentu
40) public static NodeDLL_2511533019 insertAtPosition_3019
41) (NodeDLL_2511533019 head_3019, int pos_3019, int
42) new_data_3019) {
43)     // buat node baru
44)     NodeDLL_2511533019 new_node_3019 = new
45)     NodeDLL_2511533019 (new_data_3019);
46)     if (pos_3019 == 1) {
47)         new_node_3019.next_3019 = head_3019;
48)         if (head_3019!= null) {
49)             head_3019.prev_3019 = new_node_3019;}
```

```

41)         head_3019 = new_node_3019;
42)         return head_3019; }
43)         NodeDLL_2511533019 curr_3019 = head_3019;
44)         for (int i_3019 = 1; i_3019 < pos_3019 - 1 &&
curr_3019 != null; ++i_3019) {
45)             curr_3019 = curr_3019.next_3019;
46)         }
47)         if (curr_3019 == null) {
48)             System.out.println("Posisi tidak ada");
49)             return head_3019;
50)         }
51)         new_node_3019.prev_3019 = curr_3019;
52)         new_node_3019.next_3019 = curr_3019.next_3019;
53)         curr_3019.next_3019 = new_node_3019;
54)         if (new_node_3019.next_3019 != null) {
55)
new_node_3019.next_3019.prev_3019 = new_node_3019; }
56)         return head_3019;
57) }
58) public static void printlist_3019(NodeDLL_2511533019
head_3019) {
59)     NodeDLL_2511533019 curr_3019 = head_3019;
60)     while (curr_3019 != null) {
61)         System.out.print(curr_3019.data_3019 + " <->
");
62)         curr_3019 = curr_3019.next_3019;
63)     }
64)     System.out.println();
65) }
66) public static void main(String[] args) {
67)     //membuat DLL 2 <-> 3 <-> 5
68)     NodeDLL_2511533019 head_3019 = new
NodeDLL_2511533019(2);
69)     head_3019.next_3019 = new NodeDLL_2511533019(3);
70)     head_3019.next_3019.prev_3019 = head_3019;
71)     head_3019.next_3019.next_3019 = new
NodeDLL_2511533019(5);
72)     head_3019.next_3019.next_3019.prev_3019 =
head_3019.next_3019;
73)     //cetak DLL awal
74)     System.out.print("DLL Awal : ");
75)     printlist_3019(head_3019);
76)     // tambah 1 di awal
77)     head_3019 = insertBegin_3019(head_3019, 1) ;
78)     System.out.print ("simpul 1 ditambah di awal:
");
79)     printlist_3019(head_3019) ;
80)     // tambah 6 di akhir
81)     System.out.print ("simpul 6 ditambah di akhir: ");
82)     int data_3019 = 6;
83)     head_3019 = insertEnd_3019 (head_3019, data_3019);
84)     printlist_3019(head_3019);
85)     // menambah node 4 di posisi 4
86)     System.out.print ("tambah node 4 di posisi 4: ");
87)     int data2_3019 = 4;
88)     int pos_3019 = 4;

```



```

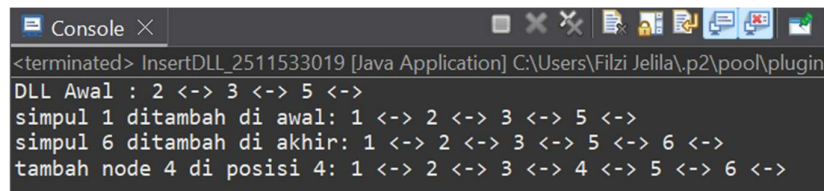
89)   head_3019 = insertAtPosition_3019(head_3019,
      pos_3019, data2_3019) ;
90)   printlist_3019(head_3019);
91)   }
92) }

```

Program kedua yaitu InsertDLL_2511533019 berfungsi untuk melakukan operasi penambahan node pada Double Linked List. Pada kelas ini terdapat beberapa method, yaitu insertBegin_3019() untuk menambahkan node di awal list, insertEnd_3019() untuk menambahkan node di akhir list, dan insertAtPosition_3019() untuk menambahkan node pada posisi tertentu.

Selain itu terdapat method printlist_3019() yang digunakan untuk menampilkan isi linked list dari awal hingga akhir. Pada method main, program membuat DLL awal berisi data 2 <-> 3 <-> 5, kemudian melakukan beberapa operasi penambahan node seperti menambah node 1 di awal, node 6 di akhir, dan node 4 pada posisi keempat. Hasil setiap proses penambahan kemudian dicetak ke layar untuk menunjukkan perubahan struktur DLL.

- 3) Run program maka akan tampil output seperti berikut.



```

<terminated> InsertDLL_2511533019 [Java Application] C:\Users\Filzi Jelila\p2\pool\plugin
DLL Awal : 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

2.1.3 Membuat program ketiga (PenelusuranDLL)

- 1) Klik kanan package pekan6 dan pilih “New”, pilih class. Lalu buat class baru dengan nama “PenelusuranDLL_2511533019.”
- 2) Lalu buat/masukkan syntax seperti berikut.

```

1) package Pekan6_2511533019;
2)
3) public class PenelusuranDLL_2511533019 {
4)     //fungsi penelusuran maju
5)     static void forwardTraversal_3019(NodeDLL_2511533019
      head_3019) {
6)         //memulai penelusuran dari head
7)         NodeDLL_2511533019 curr_3019 = head_3019;
8)         //lanjutkan sampai akhir
9)         while(curr_3019 != null) {

```

```

10)          //print data
11)          System.out.print(curr_3019.data_3019 +
" <-> ");
12)          //pindah ke node berikutnya
13)          curr_3019 = curr_3019.next_3019;
14)      }
15)      //print spasi
16)      System.out.println();
17)  }
18)  //fungsi penelusuran mundur
19)  static void
backwardTraversal_3019(NodeDLL_2511533019 tail_3019) {
20)      // mulai dari akhir
21)      NodeDLL_2511533019 curr_3019 = tail_3019;
22)      //lanjut sampai head
23)      while (curr_3019!= null) {
24)          //cetak data
25)          System.out.print(curr_3019.data_3019 +
" <-> ");
26)          // pindah ke node sebelumnya
27)          curr_3019 = curr_3019.prev_3019;
28)      }
29)      //cetak spasi
30)      System.out.println();
31)  }
32)  public static void main(String[] args) {
33)      // TODO Auto-generated method stub
34)      //cetak DLL
35)      NodeDLL_2511533019 head_3019 = new
NodeDLL_2511533019(1);
36)      NodeDLL_2511533019 second_3019 = new
NodeDLL_2511533019(2);
37)      NodeDLL_2511533019 third_3019 = new
NodeDLL_2511533019(3);
38)
39)      head_3019.next_3019= second_3019;
40)      second_3019.prev_3019= head_3019;
41)      second_3019.next_3019= third_3019;
42)      third_3019.prev_3019= second_3019;
43)
44)      System.out.println("Penelusuran maju: ");
45)      forwardTraversal_3019(head_3019);
46)      System.out.println("Penelusuran mundur: ");
47)      backwardTraversal_3019(third_3019);
48)  }
49)
50) }

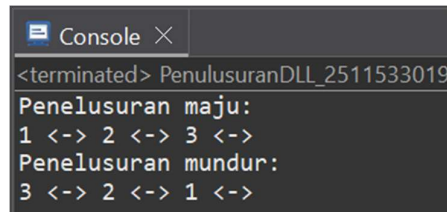
```

Program ketiga yaitu **PenelusuranDLL_2511533019** digunakan untuk melakukan penelusuran atau traversal pada Double Linked List. Terdapat dua method utama, yaitu `forwardTraversal_3019()` untuk menelusuri node dari depan ke belakang menggunakan pointer

next_3019, dan backwardTraversal_3019() untuk menelusuri node dari belakang ke depan menggunakan pointer prev_3019.

Pada method main, program membuat DLL sederhana dengan data 1 <-> 2 <-> 3, kemudian melakukan penelusuran maju dimulai dari head dan penelusuran mundur dimulai dari tail. Hasil traversal ditampilkan ke layar sehingga pengguna dapat melihat bahwa DLL dapat ditelusuri dalam dua arah.

- 3) Run program hingga keluar output seperti berikut, apabila tidak seperti pada gambar, cek kesalahan pada syntax/ kekurangan syntaxnya, dan perbaiki.



```
<terminated> PenelusuranDLL_2511533019
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Ketika program dirun, pada awalnya dibuat Double Linked List dengan isi 1 <-> 2 <-> 3. Setelah itu program melakukan penelusuran maju menggunakan method forwardTraversal_3019(). Penelusuran dimulai dari head lalu bergerak ke node berikutnya menggunakan pointer next_3019 hingga akhir list, sehingga output yang tampil adalah 1 <-> 2 <-> 3.

Selanjutnya program melakukan penelusuran mundur menggunakan method backwardTraversal_3019(). Penelusuran dimulai dari tail lalu bergerak ke node sebelumnya menggunakan pointer prev_3019, sehingga output yang dihasilkan menjadi 3 <-> 2 <-> 1. Hal ini menunjukkan bahwa Double Linked List dapat ditelusuri dari dua arah.

2.1.4 Membuat program keempat (HapusDLL)

- 1) Klik kanan package pekan6 dan pilih “New”, pilih class. Lalu buat nama “HapusDLL_2511533019”.
- 2) Masukkan syntax seperti berikut.

```
1) package Pekan6_2511533019;
```

```

2)
3) public class HapusDLL_2511533019 {
4)     //fungsi menghapus node awal
5)     public static NodeDLL_2511533019 delHead_3019
6)     (NodeDLL_2511533019 head_3019) {
7)         if (head_3019 == null) {
8)             return null;}
9)         NodeDLL_2511533019 temp_3019 = head_3019;
10)        head_3019 = head_3019.next_3019;
11)        if (head_3019!= null) {
12)            head_3019.prev_3019 = null;
13)        }
14)        return head_3019;
15)    }
16)    // fungsi menghapus di akhir
17)    public static NodeDLL_2511533019
18)    dellast_3019(NodeDLL_2511533019 head_3019) {
19)        if (head_3019 == null) {
20)            return null;
21)        }
22)        if (head_3019.next_3019 == null) {
23)            return null; }
24)        NodeDLL_2511533019 curr_3019 = head_3019;
25)        while (curr_3019.next_3019 != null) {
26)            curr_3019 = curr_3019.next_3019;
27)        }
28)        //update pointer previous node
29)        if (curr_3019.prev_3019 != null) {
30)            curr_3019.prev_3019.next_3019 = null;
31)        }
32)        return head_3019;
33)    }
34)    //fungsi menghapus node posisi tertentu
35)    public static NodeDLL_2511533019 delPos_3019
36)    (NodeDLL_2511533019 head_3019, int pos_3019) {
37)        //jika DLL kosong
38)        if(head_3019 == null) {
39)            return head_3019; }
40)        NodeDLL_2511533019 curr_3019 = head_3019;
41)        // telusuri sampai ke node yang akan dihapus
42)        for (int i_3019 = 1; curr_3019 != null &&
43)        i_3019 < pos_3019; ++i_3019) {
44)            curr_3019 = curr_3019.next_3019; }
45)        // jika posisi tidak ditemukan
46)        if (curr_3019 == null) {
47)            return head_3019;
48)        }
49)        // Update pointer
50)        if (curr_3019.prev_3019 != null) {
51)            curr_3019.prev_3019.next_3019 =
52)            curr_3019.next_3019; }
53)        if (curr_3019.next_3019 != null) {
54)            curr_3019.next_3019.prev_3019 =
55)            curr_3019.prev_3019; }
56)        //jika yang dihapus head
57)        if (head_3019 == curr_3019) {
58)            head_3019 = curr_3019.next_3019; }

```

```

53)         return head_3019;
54)     }
55)     //fungsi mencetak DLL
56)     public static void printlist_3019
57)     (NodeDLL_2511533019 head_3019) {
58)         NodeDLL_2511533019 curr_3019 = head_3019;
59)         while (curr_3019 != null) {
60)             System.out.print
61)             (curr_3019.data_3019 + " <-> ");
62)             curr_3019 = curr_3019.next_3019;
63)         }
64)         System.out.println ();
65)     }
66)     public static void main(String[] args) {
67)         // TODO Auto-generated method stub
68)         // buat sebuah DLL
69)         NodeDLL_2511533019 head_3019= new
70)         NodeDLL_2511533019(1);
71)         head_3019.next_3019 = new
72)         NodeDLL_2511533019(2);
73)         head_3019.next_3019.prev_3019 = head_3019;
74)         head_3019.next_3019.next_3019 = new
75)         NodeDLL_2511533019(3);
76)         head_3019.next_3019.next_3019.prev_3019 =
77)         head_3019.next_3019;
78)         head_3019.next_3019.next_3019.next_3019= new
79)         NodeDLL_2511533019 (4);
80)         head_3019.next_3019.next_3019.next_3019.next_3019 =
81)         head_3019.next_3019.next_3019.next_3019;
82)         System.out.print ( "DLL Awal: ");
83)         printlist_3019(head_3019);
84)         System.out.print ("Setelah head dihapus: ");
85)         head_3019 = delHead_3019(head_3019);
86)         printlist_3019 (head_3019);
87)         System.out.print ("Setelah node terakhir
88)         dihapus: ");
89)         head_3019 = delLast_3019(head_3019);
90)         printlist_3019(head_3019);
91)         System.out.print ("menghapus node ke 2: ") ;
92)         head_3019 = delPos_3019(head_3019, 2);
93)         printlist_3019(head_3019);
94)     }
95) }

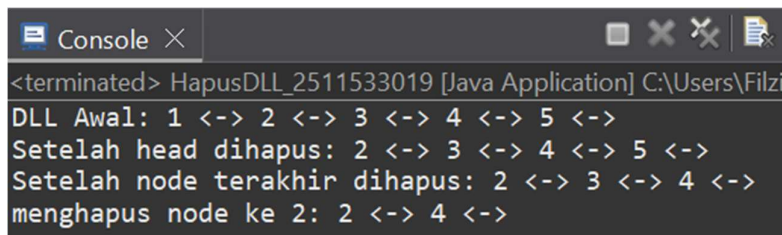
```

Program terakhir yaitu **HapusDLL_2511533019** digunakan untuk melakukan operasi penghapusan node pada Doubly Linked List.

Method `delHead_3019()` berfungsi menghapus node pertama atau head, method `delLast_3019()` digunakan untuk menghapus node terakhir, sedangkan `delPos_3019()` digunakan untuk menghapus node pada posisi tertentu.

Kelas ini juga memiliki method `printlist_3019()` untuk menampilkan isi DLL setelah proses penghapusan dilakukan. Pada method `main`, program membuat DLL dengan data 1 <-> 2 <-> 3 <-> 4 <-> 5, kemudian melakukan beberapa operasi penghapusan seperti menghapus head, menghapus node terakhir, dan menghapus node pada posisi kedua. Setelah setiap proses penghapusan, isi linked list ditampilkan untuk memperlihatkan perubahan struktur data DLL.

- 3) Run program hingga mengeluarkan output seperti berikut.



```
<terminated> HapusDLL_2511533019 [Java Application] C:\Users\Filzi
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
menghapus node ke 2: 2 <-> 4 <->
```

Setelah di run, pada awal ditampilkan linked list berisi data 1 <-> 2 <-> 3 <-> 4 <-> 5. Lalu program mencetak DLL awal tersebut. Setelah itu dilakukan penghapusan node pertama menggunakan method `delHead_3019()`, sehingga data 1 terhapus dan list berubah menjadi 2 <-> 3 <-> 4 <-> 5. Selanjutnya program menghapus node terakhir menggunakan method `delLast_3019()`, sehingga node 5 terhapus dan list menjadi 2 <-> 3 <-> 4.

Terakhir, program menghapus node pada posisi kedua menggunakan method `delPos_3019()`, sehingga node 3 dihapus dan hasil akhir DLL menjadi 2 <-> 4. Setiap hasil penghapusan ditampilkan ke layar agar perubahan struktur DLL dapat terlihat dengan jelas.

2.2 Tugas Praktikum

2.2.1 Deskripsi Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

2.2.2 Kelas Lagu

- 1) Klik kanan package pekan6 dan pilih “New”, pilih class. Lalu buat nama “**Lagu_2511533019**”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package Pekan6_2511533019;
2)
3) public class Lagu_2511533019 {
4)     String judul_3019;
5)     String penyanyi_3019;
6)     Lagu_2511533019 next_3019;
7)     Lagu_2511533019 prev_3019;
8)
9)     // konstruktor
10)    public Lagu_2511533019(String judul_3019,
    String penyanyi_3019) {
11)        this.judul_3019 = judul_3019;
12)        this.penyanyi_3019 = penyanyi_3019;
13)        this.next_3019 = null;
14)        this.prev_3019 = null;
15)    }
16)    // getter
17)    public String getJudul_3019() {
18)        return judul_3019;
19)    }
20)    public String getPenyanyi_3019() {
21)        return penyanyi_3019;
22)    }
23)    // setter
24)    public void setJudul_3019(String judul_3019)
    {
25)        this.judul_3019 = judul_3019;
26)    }
27)    public void setPenyanyi_3019(String
    penyanyi_3019) {
28)        this.penyanyi_3019 = penyanyi_3019;
29)    }
30) }
```

Program ini dibuat untuk digunakan untuk mendefinisikan node pada struktur data Double Linked List (DLL) yang merepresentasikan data lagu dalam playlist musik.

Kelas ini memiliki atribut yang sesuai dengan instruksi tugas yaitu judul_3019 untuk menyimpan judul lagu, penyanyi_3019 untuk menyimpan nama penyanyi, next_3019 sebagai pointer ke node lagu berikutnya, dan prev_3019 sebagai pointer ke node lagu sebelumnya.

Pada kelas ini terdapat konstruktor Lagu_2511533019(String judul_3019, String penyanyi_3019) yang digunakan untuk menginisialisasi data lagu saat objek dibuat, sementara pointer next_3019 dan prev_3019 diatur bernilai null karena node belum terhubung ke node lain. Selain itu terdapat method getter getJudul_3019() dan getPenyanyi_3019() untuk mengambil data judul dan penyanyi, serta method setter setJudul_3019() dan setPenyanyi_3019() untuk mengubah nilai data lagu.

Kelas ini dijadikan dasar pembentukan node pada playlist musik berbasis Double Linked List.

2.2.3 Kelas Musik

- 1) Klik kanan package pekan6 dan pilih “New”, pilih class. Lalu buat nama “**Musik_2511533019**”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package Pekan6_2511533019;
2)
3) import java.util.Scanner;
4)
5) public class Musik_2511533019 {
6)
7)     // tambah lagu di akhir
8)     public static Lagu_2511533019
       tambahLagu_3019(Lagu_2511533019 head_3019, String
       judul_3019, String penyanyi_3019) {
9)
10)         Lagu_2511533019 laguBaru_3019 = new
           Lagu_2511533019(judul_3019, penyanyi_3019);
11)
12)         // jika kosong jadi head
```



```

13)         if(head_3019 == null) {
14)             return laguBaru_3019;
15)         }
16)         // menuju node terakhir
17)         Lagu_2511533019 last_3019 = head_3019;
18)         while(last_3019.next_3019 != null) {
19)             last_3019 = last_3019.next_3019;
20)         }
21)         last_3019.next_3019 = laguBaru_3019;
22)         laguBaru_3019.prev_3019 = last_3019;
23)         return head_3019;
24)     }
25)     // hapus lagu awal
26)     public static Lagu_2511533019
hapusLaguAwal_3019(Lagu_2511533019 head_3019){
27)         if(head_3019 == null){
28)             System.out.println("Playlist
kosong");
29)             return null;
30)         }
31)
32)         System.out.println("Lagu dihapus : " +
head_3019.judul_3019);
33)         head_3019 = head_3019.next_3019;
34)
35)         if(head_3019 != null){
36)             head_3019.prev_3019 = null;
37)         }
38)         return head_3019;
39)     }
40)     // tampil maju
41)     public static void
tampilMaju_3019(Lagu_2511533019 head_3019){
42)         if(head_3019 == null){
43)             System.out.println("Playlist
kosong");
44)             return;
45)         }
46)         Lagu_2511533019 curr_3019 = head_3019;
47)         int noUrut_3019 = 1;
48)
49)         while(curr_3019 != null){
50)             System.out.println(noUrut_3019 +
". "+ curr_3019.judul_3019+ " - " +
curr_3019.penyanyi_3019);
51)             curr_3019 = curr_3019.next_3019;
52)             noUrut_3019++;
53)         }
54)     }
55)     // tampil mundur

```

```

56)         public static void
tampilMundur_3019(Lagu_2511533019 head_3019){
57)             if(head_3019 == null){
58)                 System.out.println("Playlist
kosong");
59)                 return;
60)             }
61)             Lagu_2511533019 curr_3019 = head_3019;
62)             int noUrut_3019=1;
63)
64)             while(curr_3019.next_3019 != null){
65)                 curr_3019 = curr_3019.next_3019;
66)             }
67)             while(curr_3019 != null){
68)                 System.out.println(noUrut_3019+
" . " +curr_3019.judul_3019 + " - " +
curr_3019.penyanyi_3019);
69)                 curr_3019 = curr_3019.prev_3019;
70)                 noUrut_3019++;}
71)         }
72)         // cari lagu
73)         public static boolean
cariLagu_3019(Lagu_2511533019 head_3019, String
cari_3019){
74)             Lagu_2511533019 curr_3019 = head_3019;
75)             while(curr_3019 != null){
76)
77)                 if(curr_3019.judul_3019.equalsIgnoreCase(cari
_3019)){
78)                     System.out.println("Lagu
ditemukan");
79)                     System.out.println(curr_3019.judul_3019 + " -
" + curr_3019.penyanyi_3019);
80)                     return true;
81)                 }
82)                 curr_3019 = curr_3019.next_3019;
83)             }
84)             System.out.println("Lagu tidak
ditemukan");
85)             return false;
86)         }
87)         public static void main(String[] args) {
88)             Scanner input_3019 = new
Scanner(System.in);
89)             Lagu_2511533019 head_3019 = null;
90)             int pilih_3019;
91)             do {
92)                 System.out.println("\n===
Playlist Musik NIM : 2511533019 ===");

```

```

93)         System.out.println("1. Tambah
    Lagu");
94)         System.out.println("2. Hapus
    Lagu Pertama");
95)         System.out.println("3. Lihat
    Playlist (Maju)");
96)         System.out.println("4. Lihat
    Playlist (Mundur)");
97)         System.out.println("5. Cari
    Lagu");
98)         System.out.println("6. Keluar");
99)         System.out.print("Pilihan : ");
100)        pilih_3019 =
    input_3019.nextInt();
101)        input_3019.nextLine();
102)
103)        switch(pilih_3019){
104)            case 1:
105)
106)                System.out.print("Judul : ");
107)                String judul_3019 =
    input_3019.nextLine();
108)                System.out.print("Penyanyi : ");
109)                String
    penyanyi_3019 = input_3019.nextLine();
110)                head_3019 =
    tambahLagu_3019(head_3019, judul_3019,
    penyanyi_3019);
111)                System.out.println("Lagu berhasil
    ditambahkan");
112)                break;
113)            case 2:
114)                head_3019 =
    hapusLaguAwal_3019(head_3019);
115)                break;
116)            case 3:
117)                tampilMaju_3019(head_3019);
118)                break;
119)            case 4:
120)                tampilMundur_3019(head_3019);
121)                break;
122)            case 5:
123)                System.out.print("Masukkan judul lagu : ");
124)                String cari_3019 =
    input_3019.nextLine();

```

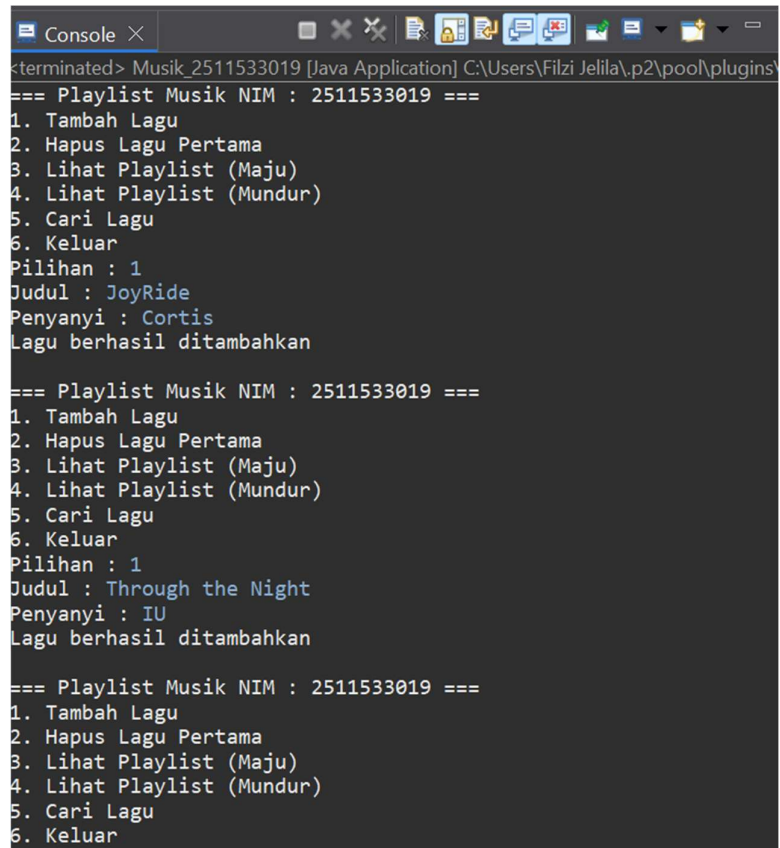
```

124)         cariLagu_3019(head_3019, cari_3019);
125)                 break;
126)         case 6:
127)
128)             System.out.println("Program selesai");
129)                 break;
130)         default:
131)             System.out.println("Pilihan salah");
132)         }
133)     } while(pilih_3019 != 6);
134)     input_3019.close();
135) }
136) }

```

Kelas Musik_2511533019 dibuat dengan menyesuaikan kebutuhan tugas playlist musik menggunakan konsep DLL. Di kelas ini terdapat Method tambahLagu_3019() dibuat berdasarkan konsep penambahan node di akhir (*insert end*) seperti praktikum DLL sebelumnya, hapusLaguAwal_3019() menggunakan konsep penghapusan head (*delete head*), sedangkan tampilMaju_3019() dan tampilMundur_3019() mengadaptasi penelusuran maju dan mundur pada DLL. Selain itu ditambahkan method cariLagu_3019() untuk mencari data lagu berdasarkan judul. Pada method main, program menggunakan menu interaktif agar user dapat menambah lagu, menghapus lagu, menampilkan playlist dari dua arah, dan mencari lagu sesuai ketentuan tugas.

- 3) Run program , berikut contoh output dengan berbagai input yang saya masukkan sebagai user:



```
<terminated> Musik_2511533019 [Java Application] C:\Users\Filzi Jelila\p2\pool\plugins\'
=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul : JoyRide
Penyanyi : Cortis
Lagu berhasil ditambahkan

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul : Through the Night
Penyanyi : IU
Lagu berhasil ditambahkan

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
```

```
Console X
<terminated> Musik_2511533019 [Java Application] C:\Users\Filzi Jelila\p2\pool\plugins\
Pilihan : 1
Judul : That Somebody
Penyanyi : Rafi Sudirman
Lagu berhasil ditambahkan

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 3
1. JoyRide - Cortis
2. Through the Night - IU
3. That Somebody - Rafi Sudirman

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu dihapus : JoyRide

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
```

```
Console X
<terminated> Musik_2511533019 [Java Application] C:\Users\Filzi Jelila\p2\pool\plugins\
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 4
1 . That Somebody - Rafi Sudirman
2 . Through the Night - IU

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 5
Masukkan judul lagu : That Somebody
Lagu ditemukan
That Somebody - Rafi Sudirman

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu dihapus : Through the Night
```

```
=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu dihapus : That Somebody

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 3
Playlist kosong

=== Playlist Musik NIM : 2511533019 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 6
Program selesai
```

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Double Linked List (DLL) merupakan struktur data dinamis yang terdiri dari node-node yang saling terhubung menggunakan dua pointer, yaitu pointer menuju node berikutnya (*next*) dan pointer menuju node sebelumnya (*previous*).

Pada praktikum ini berhasil diimplementasikan berbagai operasi dasar pada DLL seperti traversal atau penelusuran data secara maju dan mundur, penambahan node di awal, akhir, maupun posisi tertentu, serta penghapusan node pada bagian awal, akhir, dan posisi tertentu. Melalui implementasi tersebut, mahasiswa dapat memahami pengelolaan data secara dinamis serta kelebihan DLL dalam melakukan penelusuran dua arah dibandingkan Single Linked List.

Selain itu, praktikum ini juga menunjukkan bahwa DLL dapat diterapkan dalam kasus sederhana di kehidupan nyata, salah satunya pada simulasi playlist musik. Pada program tersebut, konsep DLL digunakan untuk mengelola daftar lagu mulai dari proses penambahan lagu, penghapusan lagu, pencarian lagu, serta menampilkan playlist dari arah maju maupun mundur. Implementasi ini membantu mahasiswa memahami hubungan antara konsep struktur data dengan penerapannya dalam pemrograman berbasis objek menggunakan bahasa Java.

Secara keseluruhan, praktikum ini membantu meningkatkan pemahaman mengenai konsep dasar Double Linked List, melatih kemampuan implementasi struktur data dalam bahasa pemrograman Java, serta mengembangkan logika berpikir mahasiswa dalam menyelesaikan permasalahan pengolahan data secara dinamis, terstruktur, dan lebih fleksibel melalui mekanisme penelusuran dua arah.

DAFTAR PUSTAKA

- [1] Wahyudi, “*Linked List*” PowerPoint slides, Praktikum Struktur Data, Universitas Andalas, Padang, 2025.
- [2] Oracle, “LinkedList (Java Platform SE 8),” [Online]. Available:
<https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html>